

Guide complet de soutenance – LO07 BlaBlaCar 2026

Partie de Moss'Ab : les couches **Contrôleur** et **Modèle**

Moss'Ab Mirande-Ney (binôme : Pierre Bheidi)

Table des matières

1	Contexte du projet (à connaître absolument)	1
2	Les concepts que je dois savoir expliquer	1
3	Ma partie en détail : Contrôleurs et Modèles	1
3.1	Le point d'entrée : index.php	2
3.2	Le routeur (app/router/router.php)	2
3.3	Le contrôleur de base (Controller.php)	2
3.4	Les contrôleurs concrets	2
3.5	La logique métier la plus importante : la clôture (C5)	2
3.6	Les modèles (app/model/)	3
3.7	La connexion (Database.php) et la sécurité	3
3.8	La configuration (config.php)	3
4	Déroulé d'une requête, de bout en bout	3
5	Script de démo (ce que je dis à l'écran)	3
6	Questions potentielles du jury (et réponses)	3

Ce document est un guide complet : même sans avoir codé cette partie, tu pourras tout présenter et répondre aux questions du jury. Lis-le une fois en entier, puis relis le « script de démo » et les « questions/réponses ».

1 Contexte du projet (à connaître absolument)

Le sujet : BlaBlaCar est une application web de **covoiturage**. Selon son **rôle**, l'utilisateur connecté accède à des fonctions différentes : administrateur (gère utilisateurs/véhicules/villes), conducteur (gère ses véhicules et trajets), passager (réserve et consulte).

La base de données : 5 tables : **utilisateur** (id, nom, prenom, role, login, password, solde), **vehicule** (+ propriétaire), **ville**, **trajet** (départ, arrivée, conducteur, véhicule, prix, date, heure, statut actif/passif), **reservation** (lie un trajet et un passager ; un passager peut réserver plusieurs fois).

L'architecture : Patron MVC. **Ma partie, c'est le Contrôleur et le Modèle** – le cœur logique de l'application et l'accès à la base.

2 Les concepts que je dois savoir expliquer

MVC : Le **Modèle** parle à la base de données. Le **Contrôleur** reçoit la requête de l'utilisateur, appelle le modèle, applique la logique, puis choisit la vue. La **Vue** affiche. Cette séparation rend le code clair et maintenable.

Front Controller : Un **point d'entrée unique** (`index.php`) par lequel passent toutes les requêtes. Il initialise tout (session, config, routeur) une seule fois.

Routeur : Un aiguilleur : il lit l'**action** demandée dans l'URL (ex. `?action=admin_utilisateurs`) et appelle le bon contrôleur et la bonne méthode.

PDO : *PHP Data Objects* : l'interface standard de PHP pour parler à une base de données (ici MySQL). On l'utilise avec des **requêtes préparées**.

Requête préparée : Une requête où les valeurs sont passées séparément, via des « ? ». Cela empêche les **injections SQL** : un utilisateur ne peut pas glisser du code SQL dans un champ de formulaire.

Session : Mémoire serveur qui retient l'utilisateur connecté (`$_SESSION['login_id']`) entre les pages.

Singleton : Un objet créé **une seule fois** et réutilisé partout. On l'emploie pour la connexion à la base : une seule connexion par requête HTTP.

3 Ma partie en détail : Contrôleurs et Modèles

3.1 Le point d'entrée : `index.php`

Démarrage la session, charge la config, les modèles et le routeur, puis aiguille. Au **premier accès** d'une session, on remet `login_id` à -1 pour garantir que personne n'est connecté au démarrage (exigence du sujet), sans déconnecter l'utilisateur à chaque page grâce à un drapeau de session.

```
if (!isset($_SESSION['app_demarree'])) {
    $_SESSION['app_demarree'] = true;
    $_SESSION['login_id'] = -1;
}
Router::dispatch($action);
```

3.2 Le routeur (`app/router/router.php`)

Une simple table associative action → [Contrôleur, méthode]. La méthode `dispatch()` instancie le contrôleur et appelle la méthode. Si l'action est inconnue, on renvoie l'accueil.

```
'admin_utilisateurs' => ['AdminController', 'listeUtilisateurs'], // A1
'cond_cloturer'      => ['ConducteurController', 'cloturerTrajet'], // C5
'pass_reserveur'     => ['PassagerController', 'reserveur'],      // P2
```

3.3 Le contrôleur de base (`Controller.php`)

Tous les contrôleurs en héritent. Il fournit :

- `render($vue, $data)` : affiche une vue dans le gabarit commun ;
- `redirect($action)` : redirige vers une autre action ;
- `exigerRole($roles)` : le contrôle d'accès (garde) ;
- `flash()` : messages temporaires (succès/erreur).

```
protected function exigeRole(array $rolesAutorises): array {
    $user = $this->utilisateurConnecte();
    if ($user === null || !in_array($user['role'], $rolesAutorises, true)) {
        $this->redirect('login'); // acces refuse
    }
    return $user;
}
```

```
}
```

3.4 Les contrôleurs concrets

- `AuthController` : connexion (F1) et déconnexion (F2).
- `AdminController` : A1 à A7 (utilisateurs, véhicules, villes).
- `ConducteurController` : C1 à C5 (véhicules, trajets, passagers, clôture).
- `PassagerController` : P1, P2 (réservations).
- `ExamineurController` : E1 (superglobales), E2 (10 réservations aléatoires).
- `InnovationController` : tableau de bord (données) et note MVC.

3.5 La logique métier la plus importante : la clôture (C5)

Quand un conducteur clôture un trajet : on vérifie qu'il lui appartient et qu'il est actif, on règle les **paiements** (chaque réservation débite le passager et crédite le conducteur), puis on passe le trajet en **passif**.

```
foreach ($reservations as $resa) {  
    $utilisateurModel->ajusterSolde($resa['passager_id'], -$prix); // debit  
        passager  
    $utilisateurModel->ajusterSolde($user['id'], $prix);           // credit  
        conducteur  
}  
$trajetModel->cloturer($trajetId); // statut -> passif
```

3.6 Les modèles (app/model/)

- `Model` (abstrait) : centralise PDO et les helpers `fetchAll`, `fetchOne`, `execute`, `nextId`. **Seule cette couche écrit du SQL.**
- `Utilisateur` : authentification, ajout, ajustement de solde.
- `Vehicule`, `Ville` : listes et ajouts.
- `Trajet` : requêtes avec **jointures** (noms des villes, du conducteur, du véhicule), création, clôture.
- `Reservation` : réservations d'un passager, passagers d'un trajet.

3.7 La connexion (Database.php) et la sécurité

La connexion PDO est un **singleton** (une instance partagée). PDO est configuré pour lever des exceptions. **Toutes** les requêtes sont préparées :

```
$this->fetchOne(  
    'SELECT * FROM utilisateur WHERE login = ? AND password = ?',  
    [$login, $password]  
);
```

3.8 La configuration (config.php)

Détecte l'environnement et choisit les bons identifiants de base : variables d'environnement (déploiement Docker), serveur dev-isi, ou poste local. Les mots de passe réels sont dans un fichier séparé non publié.

4 Déroulé d'une requête, de bout en bout

1. L'utilisateur appelle `index.php?action=cond_cloturer` (POST).
2. `index.php` démarre la session et appelle `Router::dispatch`.

3. Le routeur appelle `ConducteurController::cloturerTrajet()`.
4. Le contrôleur vérifie le rôle (`exigerRole`), lit le trajet et ses réservations via les modèles, effectue les paiements, clôture le trajet.
5. Il redirige vers la liste des trajets avec un message flash de confirmation.

5 Script de démo (ce que je dis à l'écran)

« Je me connecte en administrateur : voici l'ajout d'un conducteur, d'un véhicule, d'une ville – chaque insertion passe par un contrôleur qui valide les données puis appelle le modèle. *[Connexion conducteur.]* Je crée un trajet, puis je le clôture : observez les soldes – le passager est débité du prix, et mon compte de conducteur est crédité d'autant. C'est la logique métier la plus intéressante. *[Menu Examineur.]* Voici les superglobales (cookies et session), et l'ajout de 10 réservations aléatoires sur des trajets actifs. Tout l'accès à la base se fait par des requêtes préparées, dans la couche Modèle uniquement. »

6 Questions potentielles du jury (et réponses)

Q : C'est quoi un Front Controller ?

R : Un point d'entrée unique (`index.php`) par lequel passent toutes les requêtes. Il centralise l'initialisation et évite de la dupliquer dans chaque page.

Q : Comment fonctionne votre routeur ?

R : Il contient une table action $\rightarrow$ [Contrôleur, méthode]. La méthode `dispatch()` lit l'action de l'URL, instancie le bon contrôleur et appelle la méthode. Action inconnue $\rightarrow$ accueil.

Q : Comment gérez-vous l'authentification et les rôles ?

R : Après connexion réussie, on stocke l'id en session. Chaque contrôleur protégé appelle `exigerRole()` qui vérifie le rôle de l'utilisateur connecté ; sinon redirection vers le login.

Q : Pourquoi des requêtes préparées ?

R : Pour empêcher les injections SQL : les valeurs sont transmises séparément de la requête, donc non interprétées comme du code.

Q : Pourquoi un singleton pour PDO ?

R : Pour n'ouvrir qu'une seule connexion par requête HTTP et la partager entre tous les modèles.

Q : Décrivez précisément la clôture d'un trajet.

R : On vérifie que le trajet appartient au conducteur connecté et qu'il est actif. Pour chaque réservation, on débite le passager du prix et on crédite le conducteur. Enfin on passe le trajet en statut passif.

Q : Et si un passager a réservé deux fois le même trajet ?

R : Il est facturé deux fois : on parcourt toutes les lignes de réservation, donc deux réservations entraînent deux débits. C'est le comportement attendu.

Q : Pourquoi `nextId` au lieu d'`AUTO_INCREMENT` ?

R : Le jeu de données fourni insère les id manuellement. Pour rester cohérent, on calcule `MAX(id)+1` lors d'un ajout.

Q : Différence entre `fetchAll` et `fetchOne` ?

R : `fetchAll` renvoie toutes les lignes ; `fetchOne` renvoie la première (ou null). On choisit selon qu'on attend une liste ou un seul enregistrement.

Q : Comment le modèle communique-t-il avec la vue ?

R : Indirectement : le contrôleur récupère les données du modèle et les passe à la vue via `render()`. Le modèle ignore tout de l’affichage.

Q : Comment empêchez-vous un conducteur de clôturer le trajet d’un autre ?

R : Avant l’action, on vérifie que le `conducteur_id` du trajet est bien celui de l’utilisateur connecté ; sinon l’action est refusée.

Q : Avez-vous utilisé une transaction pour les paiements ?

R : Pas dans cette version : les mises à jour sont successives. Une amélioration serait de les encadrer par une transaction PDO (`beginTransaction/commit`) pour garantir l’atomicité – c’est un axe d’évolution qu’on peut citer.

Q : Qu’est-ce qu’une jointure et où l’utilisez-vous ?

R : Une requête qui combine plusieurs tables. On l’utilise dans le modèle `Trajet` pour afficher les noms des villes, du conducteur et du véhicule plutôt que leurs identifiants.

Q : Comment l’application s’adapte-t-elle au serveur (local / dev-isi) ?

R : Le fichier de configuration détecte l’environnement (variables d’env, ou présence de `dev-isi.utt.fr` dans l’URL) et choisit automatiquement les bons identifiants de base de données.